Quality of Service in Wireless Networks

Sarah Aaghbar
An-Najah National University

***Abstract*—Wireless local area networks carry voice, video, and best-effort traffic with different service requirements. This paper evaluates a Weighted Deficit Round Robin (WDRR) scheduler for a shared access-point bottleneck and compares it with first-in, first-out (FIFO) and strict-priority scheduling using a Python discrete-event simulation. Ten random-seed replications are performed for each configuration, while link capacity and buffer size are varied. At the overloaded baseline, WDRR reduces voice and video delay relative to FIFO while avoiding strict priority's most extreme best-effort delay; packet-loss ratios remain similar across the three policies. Increasing link capacity reduces congestion, whereas increasing buffer size trades fewer drops for longer delays. The model isolates queue scheduling and does not reproduce the contention, retransmissions, or channel variation of a complete IEEE 802.11 EDCA network.**

I. Introduction

Wireless local area networks (WLANs) carry many types of traffic over a shared radio channel. Voice calls and interactive video are sensitive to delay, while file transfers and background downloads can often tolerate longer waiting times. When these applications compete for limited wireless capacity, a single service policy may not meet the needs of every traffic class. Quality of service (QoS) provides mechanisms for classifying traffic and differentiating how network resources are assigned.

QoS in wireless networks is challenging because the radio channel is shared and its available capacity can change with contention, interference, retransmissions, and signal conditions. IEEE 802.11 provides QoS mechanisms, including Enhanced Distributed Channel Access (EDCA), which uses access categories and contention parameters to influence access to the medium. Traffic markings may also need to be mapped consistently between wired DiffServ networks and Wi-Fi user priorities [1], [2]. These mechanisms provide differentiated access, but they do not make wireless service capacity unlimited or guarantee that every packet meets a strict delay bound.

This project addresses the congestion problem by designing and evaluating a Weighted Deficit Round Robin (WDRR) scheduler. WDRR uses separate queues and configurable service weights to favor delay-sensitive voice and video traffic while preserving a service share for best-effort traffic. The proposed scheduler is compared with FIFO and strict-priority reference policies in a reproducible Python discrete-event simulation. Performance is evaluated using packet delay, packet loss, and delivered goodput, while link capacity and buffer size are varied to study the effect of congestion. The simulation isolates queue scheduling and is not intended to represent the full channel contention behavior of IEEE 802.11 EDCA.

The project has three objectives:

- Explain the main QoS concepts and mechanisms used in WLANs, including traffic classes, performance metrics, DiffServ mapping, and EDCA.
- Implement the proposed WDRR scheduler and compare it with FIFO and strict-priority reference policies under controlled traffic loads.
- Analyze how scheduling, link capacity, and buffer size affect delay, loss, throughput, and service differentiation.

A. Research Questions and Evaluation Logic

The project is guided by three questions. First, how does class-specific delay change when the scheduler changes from arrival-order service to strict priority or WDRR? Second, does a scheduling policy change the total amount of congestion, or mainly redistribute the delay and delivered service among classes? Third, how do capacity and buffer limits change the observed trade-off? The questions are answered with class-specific mean and 95th-percentile delay, packet-loss ratio, and delivered payload goodput.

The analysis is descriptive. The project reports averages over ten random-seed replications, and the baseline table includes standard deviations for mean delay. No hypothesis test is used to claim statistical significance. The seed-level standard deviations describe run-to-run variation in this configured model; they do not establish that a small numerical difference generalizes to another workload or wireless implementation.

B. Scope, Contributions, and Organization

The contribution is a compact, inspectable comparison of three queue schedulers under a common workload, together with capacity and buffer sensitivity results. The design pairs policies on the same packet-arrival trace, keeps the service model fixed within each scenario, and records per-class outcomes. This makes the source of the observed delay differences easier to interpret than a comparison that changes the scheduler, traffic, capacity, and buffer simultaneously.

The report also separates standards background from simulation claims. The IEEE 802.11 and DiffServ material explains the real-network context; the code models a single AP bottleneck as a finite-buffer server. It is not a standards-conformance study, a radio propagation experiment, or a measurement of EDCA. This boundary is important: terms such as "voice priority" describe the class order in the queueing model and should not be interpreted as a direct IEEE access-category configuration.

The paper proceeds as follows. Section II reviews the relevant service models and standards before defining traffic, queueing, and metrics. Section III specifies the compared scheduling policies and the WDRR configuration. Section IV describes

the experiment and reports baseline and sensitivity results. The conclusions summarize the findings and limitations. The appendices provide a compact configuration record, data interpretation notes, and the source listing for the baseline implementation.

II. System Model and Problem Description

A. Literature and Standards Context

Quality of service is best understood across several layers. Application requirements describe what a user can tolerate; IP-layer classification assigns packets to service classes; link-layer mechanisms determine access to a physical medium; and queueing policies distribute service among packets already admitted to a device. The study in this paper concentrates on the last of these. This subsection reviews the related concepts so that the measured results can be placed within their proper scope.

1) Application-Level Service Requirements: Applications differ in their tolerance for delay, delay variation, and loss. Interactive voice exchanges short packets at a relatively regular rate and is sensitive to conversational delay. Interactive video can have a similar timing sensitivity while also generating larger and more variable packets. Buffered video is more elastic: a receiver can absorb some rate variation in a playback buffer. Bulk data transfer usually prioritizes completion and sustained goodput over low per-packet latency. These distinctions explain why the single label "video" does not define one universal network requirement.

ITU-T Recommendation G.1010 organizes multimedia services by end-user quality requirements, while DiffServ service-class guidance distinguishes telephony, interactive multimedia, streaming, and multiple data classes [3], [4]. Both documents make clear that service classification should follow the application behavior and service objective. The three traffic classes used in this project are deliberately coarser. They allow a controlled experiment but should not be read as complete application profiles or service-level agreements.

2) Differentiated Services and the Wireless Boundary: Differentiated Services (DiffServ) uses a finite set of packet markings and per-hop behaviors to aggregate traffic into classes. A DSCP marking is a policy signal within a domain; it does not itself reserve bandwidth or airtime. Edge devices classify and condition traffic, and network nodes apply configured forwarding behavior to each class [4]–[6]. This aggregation avoids maintaining per-flow state at every router, but it requires consistent configuration and a policy for untrusted markings.

The transition from a wired DiffServ domain to Wi-Fi creates an additional mapping step. RFC 8325 describes mappings between DiffServ service classes and IEEE 802.11 user priorities and discusses the consequences of inconsistent treatment across that boundary [2]. A packet can be properly marked in an IP network and still receive a different class of service after entering a wireless network if the access point applies a different mapping. Conversely, accepting elevated markings from every endpoint can allow bulk traffic to consume resources intended for real-time media.

This project begins after classification. The simulator receives a class label for each packet and does not inspect DSCP, select a user priority, or enforce a trust boundary. That choice keeps the experiment focused on scheduling, but excludes classification errors, mapping policy, policing, and admission control from the measured results. In a deployed system, these mechanisms are part of the complete QoS design rather than optional details.

3) IEEE 802.11 QoS and EDCA: IEEE 802.11 specifies the MAC and PHY functions of wireless LANs. QoS enhancements introduced with 802.11e include the Hybrid Coordination Function and its Enhanced Distributed Channel Access method; QoS behavior is incorporated in the later consolidated standard family [1], [7]. EDCA defines contention-based access categories, commonly voice, video, best effort, and background. Category-specific parameters influence how long a queue waits before it can contend and how much transmission opportunity it may use after winning access.

The word "priority" has different meanings in a deterministic queue and in EDCA. In a strict-priority queue, the server selects the highest-priority nonempty queue whenever service becomes available. EDCA uses distributed contention on a shared radio channel. A favorable contention configuration changes the chance and timing of access; it does not guarantee that a particular packet is selected next. Multiple stations, collisions, retransmissions, variable PHY rates, and hidden terminals can all affect the outcome.

The present simulation therefore does not implement EDCA and does not claim to measure Wi-Fi airtime or MAC delay. It models a controlled bottleneck with constant service capacity. This abstraction permits a direct comparison of packet-selection policies, while the cited standard supplies context for the wireless application. A protocol-level validation would need an 802.11 simulator or physical testbed and should report MAC counters, airtime, retry behavior, and application metrics together.

4) Packet Scheduling Families: FIFO is the simplest policy: packets share one queue and leave in arrival order. It is easy to reason about and does not require class state, but it cannot distinguish a short delay-sensitive packet from a large elastic transfer. During overload, all classes can experience the same large queueing delay even when their requirements differ.

Strict priority uses multiple queues and always serves the highest-ranked nonempty queue. It can provide very low delay to a protected class while that class's demand remains manageable. Its defining weakness is that lower queues have no unconditional service guarantee: if higher-ranked classes continuously occupy the server, lower-ranked queues can wait for a long time. Priority also does not by itself reserve buffer space, so a shared drop-tail buffer may discard packets from every class when full.

Round-robin scheduling gives each active queue turns. Basic round robin can be unfair in bytes when packet sizes differ: a turn that sends one 1200-byte packet is not equivalent to a turn that sends one 200-byte packet. Weighted round robin assigns different amounts of service to queues, but implementations based only on packet counts can still depend strongly on packet sizes. Weighted Fair Queueing approximates fluid fair sharing with per-packet finish-time calculations, at a higher implementation cost.

Deficit Round Robin was introduced as a low-complexity approximation to fair queueing. It keeps a byte deficit per queue and carries unused service forward, which handles variable packet sizes more fairly than packet-count round robin while requiring constant work per packet in the original formulation [8]. WDRR applies configured weights by scaling the quantum assigned to a queue. In this project, WDRR is evaluated as a class scheduler; its weights are experimental settings and do not constitute standardized 802.11 service guarantees.

5) Position of the Present Experiment: The standards describe mechanisms and mapping behavior, while this study measures the consequences of three queue-selection rules under one synthetic workload. The design is intentionally modest: a shared finite buffer, independent Poisson sources, fixed payload sizes, constant link rate, and non-preemptive service. Its contribution is a reproducible, class-by-class comparison that shows how delay can move between traffic classes even when aggregate capacity and generated arrivals are held constant.

The experiment does not propose a new EDCA parameter set, a new scheduling theorem, or a guarantee for deployed WLANs. Instead, it demonstrates the roles of queue order, service weights, capacity, and buffer size in an abstract model. The separation between background standards and the coded model is maintained throughout the report to prevent queueing results from being misrepresented as protocol measurements.

B. QoS Terminology and Scope

In this paper, "traffic class" means one of the three simulator labels: voice, video, and best effort. "Queueing delay" includes time waiting in the model plus packet serialization time, because the timestamp is recorded at service completion. "Packet loss" means only a packet discarded when the shared finite buffer is full. It excludes wireless errors, retries, and losses outside the simulated bottleneck. "Goodput" is the delivered payload rate over the source-generation interval. These definitions are narrower than general network QoS metrics and are used consistently in the equations and result tables.

A service differentiation result is not automatically a fairness result. A scheduler can sharply improve one class's latency while increasing another class's delay. Likewise, a packet-loss percentage can remain similar while the distribution of delay changes substantially. The report therefore keeps class-specific results visible and avoids collapsing the comparison into a single aggregate score.

C. Wireless Access Network

We consider a wireless local area network in which multiple stations communicate through a Wi-Fi access point (AP). Traffic arriving at the AP is classified into service classes and forwarded through a shared bottleneck link. The model isolates the effects of packet queueing and scheduling at this bottleneck. It does not model radio propagation, retransmissions, or distributed contention among stations. Accordingly, it represents a controlled queueing abstraction rather than a complete IEEE 802.11 MAC and PHY simulation.

TABLE I: Traffic classes in the baseline scenario.

Class	Arrival rate	Payload	Offered rate
Voice	250 packets/s	200 bytes	0.40 Mb/s
Video	240 packets/s	1000 bytes	1.92 Mb/s
Best effort	400 packets/s	1200 bytes	3.84 Mb/s
Aggregate	–	–	6.16 Mb/s

IEEE 802.11 provides QoS mechanisms, including Enhanced Distributed Channel Access (EDCA), which differentiates channel access using traffic categories and contention parameters. Consistent mapping between IP-layer traffic markings and Wi-Fi user priorities is also important for end-to-end service differentiation [1], [2]. These mechanisms provide the standards context for this study; the simulation evaluates queue scheduling separately from wireless channel contention.

D. Traffic Classes and Offered Load

The workload contains three traffic classes: voice, video, and best-effort data. Each class generates packets according to an independent Poisson arrival process, with a fixed payload size for that class. The offered bit rate of class i is

$$R_i = 8\lambda_i L_i, \tag{1}$$

where λ_i is the packet arrival rate in packets per second and L_i is the payload size in bytes.

The baseline bottleneck capacity is $C = 4.5\ \mathrm{Mb/s}$, which is lower than the aggregate offered load of $6.16\ \mathrm{Mb/s}$. The corresponding load ratio is

$$\rho = \frac{\sum_i R_i}{C} = \frac{6.16}{4.5} \approx 1.369. \tag{2}$$

Since $\rho > 1$, the workload is overloaded: the link cannot sustainably serve all offered traffic. With a finite buffer, the queue remains bounded, but packet drops occur when the buffer is full.

E. Queue and Service Model

The bottleneck is modeled as a link with constant service capacity C. A packet containing L payload bytes requires a service time of

$$S = \frac{8L}{C}, \tag{3}$$

where C is expressed in bits per second. The shared buffer holds at most 200 packets, regardless of class. An arriving packet is dropped if the buffer is full. Traffic is generated for 60 seconds; after the generation interval ends, the simulation continues until every admitted packet has completed service.

Three scheduling policies are evaluated: FIFO, strict priority, and the proposed Weighted Deficit Round Robin (WDRR) scheduler. FIFO serves all admitted packets in order of arrival. Strict priority uses separate class queues and always selects a packet from the highest-priority nonempty queue, in the order voice, video, then best effort. Service is non-preemptive: a packet already in transmission completes before the next scheduling decision.

WDRR also maintains a queue for each class, together with a deficit counter. During each visit, a class receives a byte

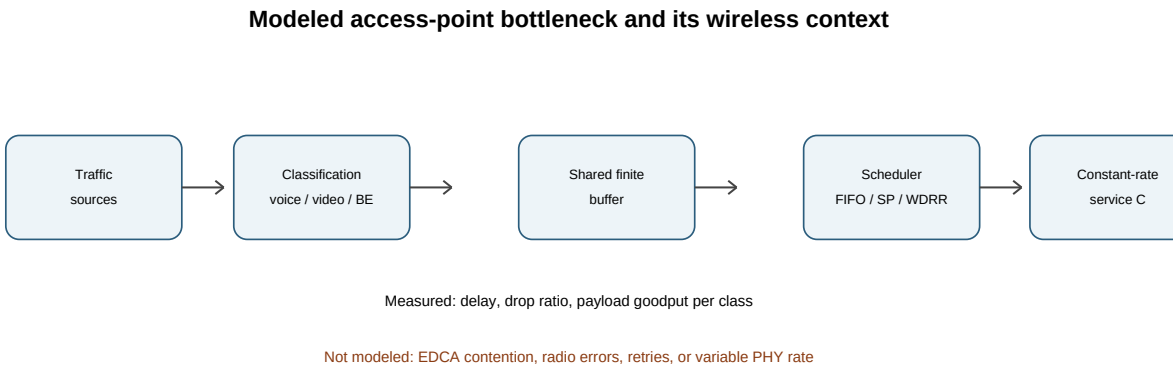


Fig. 1: Model boundary. Traffic is classified before entering the shared waiting buffer; the selected policy schedules packets for constant-rate service. The plotted metrics are measured per class. Wireless contention and radio errors are outside the simulator.

quantum proportional to its configured weight. It may transmit packets while its accumulated deficit is at least as large as the head-of-line packet size. The initial voice, video, and best-effort weights are $1 : 5 : 5$, respectively. Thus, WDRR differentiates traffic while retaining a configured service share for each active class.

All three policies use the same arrival traces, bottleneck capacity, and shared-buffer limit in each paired run. FIFO and strict priority are reference policies; WDRR is the proposed solution. None of the three is a direct implementation of IEEE 802.11 EDCA, and the results must be interpreted within the scope of the queueing model.

F. Discrete-Event Simulation Procedure

The simulator represents packet arrivals and service completions as discrete events. For each traffic class, packet arrivals are generated using an independent Poisson process over the 60-second traffic interval. For a given seed and network configuration, the same arrival trace is used by FIFO, strict-priority scheduling, and WDRR. This paired comparison keeps the input workload identical across policies.

The simulation processes the next arrival or service-completion event, whichever occurs first. At an arrival event, the packet is admitted to its class queue if the waiting buffer has space; otherwise, it is counted as dropped. If the server is idle, the scheduler selects the next packet according to the active policy and begins its non-preemptive service. At a completion event, the packet's delay and payload bytes are recorded, and the scheduler selects the next packet if one is waiting.

The event loop continues after the 60-second arrival period until every admitted packet has completed service. Thus, the observation interval includes the queue-draining period. The buffer limit applies to packets waiting in the queues; a packet currently in service is not counted against that waiting-buffer limit.

For WDRR, each class i has a deficit counter D_i and a quantum q_i. The configured quantum is proportional to the class weight:

$$q_i = 1500 w_i \quad \text{bytes}, \tag{4}$$

where w_i is the configured weight. The baseline weights are $w_{\text{voice}} = 1$, $w_{\text{video}} = 5$, and $w_{\text{best}} = 5$.

When the scheduler visits a nonempty class queue, it checks the size of the head-of-line packet. If the deficit is smaller than the packet size, the class quantum is added to the deficit and the scheduler advances to the next class. If the deficit is sufficient, the packet is transmitted and its size is subtracted from the deficit. The scheduler may serve further packets from that class while its remaining deficit can cover the next packet. When a class queue becomes empty, its deficit is reset to zero.

The experiment evaluates three link capacities (4.5, 5.0, and 6.5 Mb/s), three waiting-buffer limits (50, 200, and 500 packets), and three scheduling policies. Each configuration is repeated with ten seeds, from 2026 through 2035, for a total of 270 policy runs. For each seed and configuration, all three policies receive the same generated arrivals.

The simulation checks packet accounting by verifying that each generated packet is either delivered or dropped. It also checks that measured aggregate goodput does not exceed the configured link capacity. Both checks passed for the reported experiment runs.

G. Traffic Source Model and Load Regimes

For each traffic class i, the arrival process is modeled as a homogeneous Poisson process with rate λ_i. Equivalently, interarrival times are independent exponential random variables with mean $1/\lambda_i$:

$$\Pr\{X_i > x\} = \exp(-\lambda_i x), \qquad x \geq 0. \tag{5}$$

The expected number of class-i arrivals over a generation interval T_{gen} is $\lambda_i T_{\text{gen}}$. The actual count varies from seed to seed. The simulation records the generated count for each run instead of assuming that the observed count is exactly equal to its expectation.

The mean offered bit rate is the packet rate multiplied by the payload bits per packet. With fixed payload length L_i bytes, the rate is $R_i = 8\lambda_i L_i$. Summing over classes gives $R_{\text{off}} = 6.16$ Mb/s. The nominal load ratio $\rho = R_{\text{off}}/C$ is

approximately 1.369 at 4.5 Mb/s, 1.232 at 5.0 Mb/s, and 0.948 at 6.5 Mb/s. The first two settings are overloaded in the long-run mean; the last has nominal capacity above mean offered load. The load ratio is a mean-rate indicator and does not rule out short-term bursts when $\rho < 1$.

H. Finite Buffer, Admission, and Drop Accounting

The shared waiting buffer has a limit B measured in packets. The baseline is $B = 200$. At an arrival event, a packet is accepted if the waiting buffer contains fewer than B packets. Otherwise, that new arrival is dropped and the class-specific drop counter is incremented. The packet currently being transmitted is outside the waiting-buffer count, so the total packet population can temporarily include the buffer limit plus one packet in service. This convention is held constant across all policies.

The model uses drop-tail behavior: packets are discarded only when the shared queue reaches its limit. It does not reserve buffer space for voice or video, apply random early marking, or use a separate per-class threshold. Consequently, service priority can change which packets depart first but does not guarantee class-specific admission or loss protection. This is an important reason that baseline loss ratios remain close even though per-class delay differs sharply.

For every run and class, the accounting identity is

$$N_i^{\mathrm{gen}} = N_i^{\mathrm{del}} + N_i^{\mathrm{drop}}. \tag{6}$$

All generated packets are therefore classified as either delivered or dropped. The validation output reported that packet accounting passed. The simulator also checks that aggregate payload goodput does not exceed the configured service capacity, and that check passed for the reported runs.

I. Service-Time and Delay Decomposition

A packet with L payload bytes receives $8L$ payload bits of modeled service. At constant link rate C bits/s, its serialization time is

$$S(L) = \frac{8L}{C}. \tag{7}$$

For example, at 4.5 Mb/s, the model assigns 0.356 ms to a 200-byte voice payload, 1.778 ms to a 1000-byte video payload, and 2.133 ms to a 1200-byte best-effort payload. These values count payload only; frame headers, interframe spacing, contention, acknowledgments, retransmissions, and PHY coding are not represented.

If packet k of class i arrives at $a_{i,k}$ and completes service at $d_{i,k}$, its reported delay is $D_{i,k} = d_{i,k} - a_{i,k}$. In this single-server model, this can be separated into queue waiting $W_{i,k}$ and serialization $S_{i,k}$:

$$D_{i,k} = W_{i,k} + S_{i,k}. \tag{8}$$

Since the payload sizes and capacity are fixed within a class and scenario, serialization time is constant for that class; most differences between scheduler policies therefore arise from the waiting term. The model assumes non-preemptive service: the scheduler cannot interrupt a packet once its transmission has begun.

J. Model Assumptions and Their Consequences

Table II records the principal assumptions. Each assumption makes the experiment simpler to reproduce, but also bounds the interpretation of its results. These assumptions are appropriate for isolating queue selection. They are not interchangeable with an 802.11 MAC model. In particular, the constant-rate service abstraction removes the random medium-access delay that would otherwise interact with queue priority.

K. Performance Measures

Performance is measured separately for each traffic class. The delay of a delivered packet is its service-completion time minus its arrival time. For class i, the mean delay is

$$\overline{D}_i = \frac{1}{N_i^{\mathrm{del}}} \sum_{k=1}^{N_i^{\mathrm{del}}} D_{i,k}, \tag{9}$$

where N_i^{del} is the number of delivered packets and $D_{i,k}$ is the delay of delivered packet k.

The packet-loss ratio is

$$P_i^{\mathrm{loss}} = \frac{N_i^{\mathrm{drop}}}{N_i^{\mathrm{arr}}}, \tag{10}$$

where N_i^{drop} and N_i^{arr} denote the dropped and generated packet counts for class i. Delivered goodput is the successfully transmitted payload data per unit time:

$$G_i = \frac{8B_i^{\mathrm{del}}}{T}, \tag{11}$$

where B_i^{del} is the number of delivered payload bytes and T is the traffic-generation duration. Goodput is reported in megabits per second. These metrics are considered together because a policy may reduce delay for one class while increasing loss or reducing service for another.

III. QoS Scheduling and Proposed Solution

A. Problem Formulation

A wireless access point may carry traffic with different service requirements over a shared, capacity-limited link. In the baseline scenario, the aggregate offered load is greater than the bottleneck capacity. The network therefore cannot sustain delivery of all offered traffic without some combination of increased queueing delay, packet loss, or traffic-rate reduction. The scheduling problem is to allocate the available service so that delay-sensitive traffic receives timely service while lower-priority traffic continues to make progress.

FIFO scheduling does not distinguish traffic classes: a voice packet may wait behind a long queue of video or best-effort packets. Strict-priority scheduling can reduce this delay by serving higher-priority queues first, but sustained high-priority traffic can leave best-effort packets waiting for an excessive time. The project addresses this trade-off by proposing a weighted scheduler that differentiates traffic classes while assigning a configured service share to each active class.

TABLE II: Model assumptions and implications for interpretation.

Assumption	Implementation in the model	Consequence
Independent sources	Poisson arrivals are generated independently by class	Captures random arrivals but not correlated application bursts or trace replay
Fixed payload size	Every packet in a class has the same payload length	Separates class size effects but omits within-class packet-size variation
Constant service rate	All payload is transmitted at configured C	Excludes PHY-rate changes, retries, and contention-dependent airtime
Shared finite buffer	One packet limit applies across all queues	Policies change service order but do not reserve memory by class
Drop-tail admission	An arrival is dropped only when the waiting buffer is full	Measures queue overflow, not radio errors or transport loss
Non-preemptive service	An in-service packet completes before another is selected	Higher priority cannot interrupt an already transmitted lower-class packet
Finite source interval	Arrivals stop after 60 s and admitted packets drain	Delays include the complete queue-draining period; goodput uses the 60-s generation interval
Paired traces	Each policy receives the same arrival trace for a seed	Reduces workload variation in policy comparisons

B. Reference Scheduling Policies

1) First-In, First-Out: Under first-in, first-out (FIFO) scheduling, all admitted packets share one queue. The packet with the earliest arrival time is transmitted first, regardless of its traffic class. If packet n arrives at time a_n and completes service at time d_n, its delay is

$$D_n = d_n - a_n. \tag{12}$$

FIFO is simple and preserves arrival order. However, it provides no mechanism to protect voice or video packets from queueing behind a large volume of data traffic.

2) Strict-Priority Scheduling: Strict-priority scheduling maintains a separate queue for each traffic class and always serves the highest-priority nonempty queue. The priority order in this study is voice, video, and then best effort. Let $Q_c(t)$ be the number of packets waiting in class c at time t. With indices 1 for voice, 2 for video, and 3 for best effort, the selected class is

$$c^*(t) = \min\{c \in \{1,2,3\} : Q_c(t) > 0\}. \tag{13}$$

Service is non-preemptive: a packet already in transmission completes before the next scheduling decision. Strict priority can protect delay-sensitive packets, but persistent high-priority load can severely delay or starve lower-priority traffic.

C. Proposed Weighted Deficit Round Robin Scheduler

The proposed solution is a Weighted Deficit Round Robin (WDRR) scheduler with separate queues for voice, video, and best-effort traffic. WDRR combines class differentiation with continued service for each active queue. Each class i is assigned a weight w_i and a byte quantum

$$q_i = q_0 w_i, \tag{14}$$

where $q_0 = 1500$ bytes is the base quantum used in the simulation. During a visit to class i, its deficit counter is increased by q_i. The scheduler transmits packets from that queue while the head-of-line packet size does not exceed the available deficit, subtracting each transmitted packet's size from the counter. It then visits the next active class. A class that becomes empty has its deficit reset, and empty queues are skipped.

The initial weight configuration is

$$w_{\text{voice}} : w_{\text{video}} : w_{\text{BE}} = 1 : 5 : 5. \tag{15}$$

When all three queues remain backlogged, the approximate long-term service share of class i is

$$C_i \approx \frac{w_i}{\sum_j w_j} C, \tag{16}$$

where C is the bottleneck capacity. At $C = 4.5$ Mb/s, the selected weights correspond to approximate shares of 0.41 Mb/s for voice, 2.05 Mb/s for video, and 2.05 Mb/s for best effort under continuous backlog. These configured shares are design parameters for the experiment; they are not guarantees defined by IEEE 802.11. The results in Section IV show the delay and service trade-off they produce for the specified workload.

D. Service Allocation and Delay Trade-Offs

The three scheduling policies differ in the service decision made after a packet completes. FIFO chooses the oldest waiting packet from a common queue. Strict priority first chooses a nonempty class queue according to a fixed rank and then serves its oldest packet. WDRR chooses a class according to a cyclic visit order and its available byte deficit. The service rate, packet sizes, generated arrivals, buffer limit, and non-preemptive rule are otherwise held constant.

1) FIFO as a Neutral Reference: FIFO is class-neutral in the sense that the class label does not enter the selection rule. If two packets are admitted at times $a_m < a_n$, packet m is selected first provided both remain queued when the server is ready. This preserves arrival order and makes implementation and auditing straightforward. It does not imply equal delay under every trace: class packet sizes and arrival patterns can differ, and a packet experiences the backlog that preceded it. In this experiment, the queue has a shared packet limit. A burst can place packets from all classes behind earlier traffic. Since the voice packets are smaller than video and best-effort packets, their transmission time is shorter, but FIFO cannot

exploit this size difference to move voice packets forward. The baseline table therefore serves as the class-independent reference rather than an optimal policy.

2) Strict Priority and Starvation Risk: Let the class order be voice $\succ$ video $\succ$ best effort. Whenever the server becomes idle, strict priority selects the first nonempty queue in that order. This policy gives the selected classes the smallest waiting times among the evaluated rules, subject to the non-preemptive packet already in service. The lowest class receives any service left after higher classes are served.

Strict priority is not work-conserving across classes in the sense of reserving shares; it is work-conserving at the single server because it always serves a packet when any queue is nonempty. A lower-priority queue may be delayed for a long interval if higher-priority work continues to arrive. In the present workload, voice and video aggregate offered rate is 2.32 Mb/s, below the 4.5 Mb/s server capacity, so the lowest queue continues to receive service. However, the best-effort backlog grows because total offered load is 6.16 Mb/s. If voice and video load were raised toward capacity, the best-effort waiting time could become arbitrarily large in an infinite-buffer model or remain near the buffer horizon with losses in this finite model.

3) WDRR Deficit Accounting: WDRR maintains a queue, weight w_i, quantum q_i, and deficit counter D_i for every class i. At a visit, the scheduler adds one quantum to the class deficit. It may transmit the head packet if its byte length is no greater than the deficit; after service, the packet length is subtracted. If no packet fits, the scheduler visits another class and preserves the remaining deficit for the next round. When a class becomes empty, its deficit is reset in the implementation used here.

For the configured ratio $(1, 5, 5)$, the base quantum is 1500 bytes and the class quanta are 1500, 7500, and 7500 bytes. The selected quantum is at least as large as the maximum packet payload in each class, so a nonempty class can transmit at least its head packet during a visit after receiving a quantum. Because voice packets are 200 bytes, one voice quantum may permit several voice packets in a visit. For video and best-effort packets of 1000 and 1200 bytes, respectively, a 7500-byte quantum may permit multiple packets, subject to queue occupancy.

When all queues are continuously backlogged and service is apportioned in proportion to these byte quanta, the nominal shares are

$$\widehat{C}_i = \frac{w_i}{\sum_j w_j} C. \tag{17}$$

At 4.5 Mb/s, this gives approximately 0.409 Mb/s for voice and 2.045 Mb/s for each of video and best effort. These are idealized long-run byte shares under continuous backlog, not measured goodput, not per-packet delay bounds, and not IEEE 802.11 guarantees. Idle queues, packet boundaries, the finite run, and the workload's class-specific demand all affect the observed rates.

4) Why Packet Size Matters: A packet scheduler selects indivisible packets. A small quantum can require multiple visits before a large head packet becomes eligible. Deficit accounting carries unused bytes forward so that a class is not permanently denied service merely because its packets are larger than one quantum. This mechanism is the reason WDRR is more appropriate than packet-count round robin when classes have different payload sizes [8].

The modeled service times at the baseline are about 0.356 ms, 1.778 ms, and 2.133 ms for voice, video, and best effort. A packet already in transmission blocks every other class until completion. Thus, even strict priority cannot reduce a newly arriving voice packet's delay below the residual transmission time of the current packet plus its own service time. The measured strict-priority delays remain low because the queueing model has no random channel-access delay and because voice and video arrival rates are manageable relative to service.

5) Pseudocode for Policy Selection: The following policy sketch summarizes the decision logic. It omits event-queue bookkeeping, which is described in Section II-F.

```
FIFO:
    choose the waiting packet with the earliest a

STRICT PRIORITY:
    for class in [voice, video, best_effort]:
        if class queue is nonempty: choose its he

WDRR:
    visit the next class in cyclic order
    if its queue is empty: reset its deficit and
    otherwise add its byte quantum
    while the head packet fits the deficit:
        transmit the head packet and subtract its
        stop when the server begins the selected
    continue visits until one packet is selected
```

The pseudocode is explanatory. The implementation's complete baseline listing is included in Appendix E, and the results tables report the multi-seed and sensitivity aggregates supplied for the project.

E. Design and Deployment Implications

The experiment suggests a sequence for translating queueing results into a real QoS design. First, the network operator should identify application requirements rather than assign priority from a traffic label alone. The label "video" may represent an interactive call or a buffered stream, and these applications can have different delay and loss tolerance. Traffic classification should be associated with a documented policy and a trust boundary.

Second, priority should be paired with a load policy. If high-priority offered load can approach or exceed service capacity, strict priority can leave lower classes with excessive delay. Admission control can limit the number of real-time sessions; shaping can smooth bursts; rate adaptation can reduce video demand; and capacity planning can increase available service. These controls address the load that scheduling alone cannot remove. DiffServ guidance treats classification, conditioning, per-hop behavior, queueing, and active queue management as parts of class configuration rather than a single priority bit [4].

Third, buffer configuration should reflect delay requirements. A large queue can reduce burst drops but permit longer waits under overload. For interactive traffic, a packet arriving after its deadline may have little value even if it is eventually transmitted. Per-class queue limits, active queue management, and deadline-aware discard can complement a scheduler, but each changes the experiment and must be evaluated independently. The current project uses one shared drop-tail buffer and therefore does not quantify those alternatives.

Fourth, scheduler weights should be selected against measurable objectives. A WDRR weight ratio is a relative byte-service configuration; it is not a guarantee of application delay. Before deployment, an operator should test at least the baseline load, a bursty load, and a high-priority saturation case. Measurements should include queue occupancy, per-class delay percentiles, loss, delivered goodput, and starvation duration. In Wi-Fi, airtime and retry counters should be added because equal payload bytes can consume different airtime at different PHY rates.

Finally, the wired-to-wireless mapping should be validated end to end. A QoS marking must be classified consistently at the AP, mapped to the intended Wi-Fi user priority, placed into the intended access category, and treated according to the configured contention parameters. RFC 8325 discusses DSCP-to-user-priority mapping and the risks of inconsistent treatment across wired and wireless segments [2]. The simulator starts after this mapping, so it cannot detect a mismatch in a deployed network.

This table is a summary of the present measurements, not a universal recommendation. Another offered-load mix or packet-size distribution could change the relative outcomes. A deployment should first establish the service objectives and then verify them with representative traces and the actual Wi-Fi MAC.

F. Relationship to IEEE 802.11 EDCA

IEEE 802.11 Enhanced Distributed Channel Access (EDCA) differentiates channel access using four access categories: voice, video, best effort, and background. Categories use contention parameters, including arbitration interframe spaces and contention-window settings, to influence their opportunities to access the shared wireless medium. Unlike the deterministic queue selection used by FIFO, strict priority, or WDRR in this project, EDCA operates through distributed contention among wireless stations. Consistent mapping of IP service markings to Wi-Fi user priorities is also important [1], [2].

The proposed WDRR scheduler is evaluated as a queueing solution at a simplified access-point bottleneck. It does not implement EDCA contention, radio retransmissions, or variable channel rates. Accordingly, the results characterize the modeled scheduler and workload; they are not direct measurements or predictions of full IEEE 802.11 EDCA performance.

G. Evaluation Questions and Metrics

The experiment compares FIFO, strict priority, and WDRR using the same packet-arrival traces, link capacity, and shared buffer. It addresses three questions:

1) Does WDRR reduce mean and 95th-percentile delay for voice and video compared with FIFO?
2) Does WDRR preserve more best-effort goodput than strict priority under overload?
3) How sensitive are the results to bottleneck capacity and buffer size?

For each traffic class, delay is measured over delivered packets. Packet-loss ratio is calculated as the number of dropped packets divided by the number of generated packets. Delivered goodput is the successfully transmitted payload data per unit time. These metrics are reported together because a low delay value alone does not show how many packets were dropped, and a throughput value alone does not reveal how service was distributed among classes.

IV. Experimental Design and Analysis Method

A. Evaluation Objectives

This evaluation examines whether Weighted Deficit Round Robin (WDRR) can provide a more balanced distribution of service than FIFO and strict-priority scheduling in a congested wireless access-point bottleneck. The analysis focuses on three questions:

1) How does each scheduling policy affect packet delay, packet loss, and delivered goodput for voice, video, and best-effort traffic?
2) How do link capacity and buffer size change these performance measures?
3) Does WDRR reduce the risk of excessive service delay for lower-priority traffic while continuing to serve delay-sensitive classes?

These questions are evaluated using the discrete-event queueing model defined in Section II. The model isolates packet scheduling at a shared bottleneck. Therefore, the results describe the behavior of this queueing model and are not measurements of a physical Wi-Fi deployment.

B. Compared Scheduling Policies

Three policies were evaluated using the same traffic classes and packet arrival traces.

- **FIFO:** all admitted packets share one queue and are transmitted in arrival order.
- **Strict priority:** packets are placed in separate class queues. The scheduler serves voice before video and video before best-effort traffic.
- **WDRR:** packets are placed in separate class queues and served using the configured class weights. The baseline weight ratio is voice:video:best effort $= 1 : 5 : 5$.

The WDRR configuration is designed to differentiate service while preserving opportunities for all active classes. Its performance is assessed from the simulation measurements; the weight ratio alone does not establish a delay or loss guarantee.

C. Experiment Configuration

The baseline workload uses the arrival rates and packet sizes listed in Table I. Its aggregate offered rate is $6.16\ \mathrm{Mb/s}$. The

TABLE III: Policy selection considerations suggested by the measured trade-offs.

Policy	Observed strength	Observed cost	Suitable interpretation
FIFO	Similar service order across classes; simple to audit	Delay-sensitive classes wait behind earlier bulk traffic under overload	Neutral baseline when no class differentiation is required
Strict priority	Lowest voice and video delays in the tested configurations	Best-effort delay is highest; lower-class delay can grow sharply with buffer size	Strong preference for high classes, only with load control and starvation safeguards
WDRR	Keeps voice/video delay well below FIFO and best-effort delay below strict priority in the baseline	Does not eliminate overload; delay still depends on weights, packet sizes, and buffer length	Weighted sharing when every active queue should retain opportunities for service

TABLE IV: Simulation settings used in the experiments.

Parameter	Values
Traffic-generation duration	60 s
Scheduling policies	FIFO, strict priority, WDRR
WDRR weight ratio	$1:5:5$
Link capacity	4.5, 5.0, 6.5 Mb/s
Buffer capacity	50, **200**, 500 packets
Replications per setting	10

baseline bottleneck capacity is 4.5 Mb/s, giving an offered-load ratio of approximately 1.369. This overloaded setting was used to observe queue buildup and packet drops under a finite buffer.

To examine sensitivity to network conditions, the experiment varies link capacity and buffer size around the baseline. Table IV summarizes the settings used in the experiments. The baseline case is highlighted in bold; seed values are 2026 through 2035.

Each replication uses a fixed random seed. For a given replication and network setting, all three policies receive the same generated packet arrivals. This paired design makes the policy comparison more consistent: observed differences are less likely to result from different traffic realizations. The seed and configuration were recorded for every run to make the results reproducible.

After the traffic-generation interval ends, the simulation continues until all admitted packets have completed service. Packets that arrive while the buffer is full are counted as dropped. The same buffer-limit convention was applied to all policies.

D. Performance Metrics

Performance is reported separately for each traffic class. Let N_i^{gen} denote the number of packets generated for class i, N_i^{del} the number delivered, and N_i^{drop} the number dropped. Let $D_{i,k}$ be the end-to-end queueing-model delay of the kth delivered packet in class i.

The mean delay for class i is

$$\overline{D}_i = \frac{1}{N_i^{\mathrm{del}}} \sum_{k=1}^{N_i^{\mathrm{del}}} D_{i,k}. \tag{18}$$

Mean delay summarizes the average experience of delivered packets, but it can hide a small number of packets with very long waiting times. The simulation therefore also reports the 95th-percentile delay, $D_{i,95}$, defined as the value at or below which 95% of the delivered packet delays fall. For each seed, the percentile is calculated per run and the reported ten-seed value is the arithmetic mean of run-level percentiles, not a pooled percentile over all packets.

The packet-loss ratio for class i is

$$P_i^{\mathrm{loss}} = \frac{N_i^{\mathrm{drop}}}{N_i^{\mathrm{gen}}}. \tag{19}$$

The delivered goodput for class i is

$$G_i = \frac{8B_i^{\mathrm{del}}}{T_{\mathrm{gen}}}, \tag{20}$$

where B_i^{del} is the number of successfully delivered payload bytes and T_{gen} is the traffic-generation duration. Goodput counts delivered payload data rather than packets that were generated but subsequently dropped.

For each metric, results are summarized across the independent replications. The baseline table reports the mean and standard deviation of delay across seeds; the sensitivity tables report mean delay. Delay, loss, and goodput are interpreted together because a scheduler may improve one measure for a class while worsening another.

E. Analysis Procedure

The analysis proceeded in three stages. First, the policies were compared under the baseline capacity and buffer settings. Second, link capacity was varied while the buffer size and traffic workload were held fixed. Third, buffer size was varied while capacity and workload were held fixed. In every stage, class-specific results are examined alongside aggregate results so that gains for one class are not mistaken for improvements to the entire system.

The central comparison is between WDRR and the two reference policies. The analysis assesses whether WDRR provides a useful compromise between FIFO's class-independent service and strict priority's preference for higher-priority queues. In particular, the measurements were checked for changes in voice and video delay, best-effort delay and goodput, and packet loss in each class.

The numerical results reported in the following subsections were obtained‘nfrom the implemented simulation using the traffic, capacity, buffer, and‘nseed settings described above.

F. Baseline Results

The baseline results are averaged over ten random seeds using a link capacity of 4.5 Mb/s and a buffer capacity of 200 packets. The aggregate offered traffic is 6.16 Mb/s, which

exceeds the link capacity. The measured packet loss is therefore approximately 26.2–26.6% across the three scheduling policies.

1) Variation Across Random Seeds: The baseline table reports ten-seed means and standard deviations for mean delay. To make the scale of run-to-run variation easier to interpret, Table VI gives approximate 95% t intervals for the mean delay, computed as $\bar{x} \pm t_{0.975,9} s/\sqrt{10}$ using the reported standard deviation s. These intervals assume independent seed-level run means and are descriptive summaries. They are not hypothesis tests, and overlap or non-overlap should not be treated as a formal policy comparison.

The narrow intervals reflect low variation among the ten generated traces for this fixed, synthetic workload. They do not account for model uncertainty, parameter uncertainty, alternative application traces, or variation in real wireless environments. Sensitivity tables report means only because per-seed standard deviations for every sensitivity metric were not included in the supplied results.

2) Delay and Service Differentiation: FIFO produces similar mean delays for all three classes, approximately 302–304 ms. This reflects its class-independent service order: voice and video packets receive no scheduling preference when they share the queue with best-effort traffic.

Strict-priority scheduling substantially reduces the mean delay of voice and video to 1.458 ms and 3.319 ms, respectively. However, the mean best-effort delay rises to 671.373 ms, with a 95th-percentile delay of 724.467 ms. These results show the cost of serving higher-priority queues first when the offered load exceeds the link capacity.

WDRR provides an intermediate delay distribution. Voice and video mean delays are 23.411 ms and 19.143 ms, respectively: higher than under strict priority but much lower than under FIFO. The best-effort mean delay is 645.874 ms, about 25.5 ms lower than with strict priority in this baseline experiment. Thus, with the selected weights, WDRR reduces the delay extremes caused by strict priority while retaining substantial service differentiation.

3) Relative Delay Change by Class: Table VII expresses the baseline mean delays as percentage changes relative to FIFO. Negative values indicate lower mean delay than FIFO; positive values indicate a higher mean delay. The table is calculated from the ten-seed means and is intended to make the class trade-off visible, not to establish statistical significance.

The percentage view emphasizes that scheduler performance is class-dependent. WDRR cuts mean delay by more than 90% for voice and video compared with FIFO, yet its best-effort mean delay is more than twice the FIFO value. Strict priority produces the lowest voice and video means, but also the highest best-effort mean. An aggregate mean over all delivered packets would obscure this difference because the classes have different packet rates and packet sizes.

4) Voice-Class Interpretation: Voice has the smallest payload in the workload and the lowest offered rate of the three classes. Under FIFO, however, its mean delay is 302.366 ms and its 95th-percentile delay is 322.935 ms because the class cannot bypass packets already waiting in the shared queue. Strict priority reduces the mean to 1.458 ms and the 95th percentile to 2.381 ms. WDRR produces a mean of 23.411 ms and a 95th percentile of 45.075 ms. Thus, both class-aware policies materially improve voice delay in the simulated baseline, with strict priority producing the smallest values.

These queueing-model numbers should not be compared directly with an application deadline for a deployed call. The measurement excludes medium-access contention and radio retries, and the traffic source is a fixed-rate Poisson process rather than a codec trace. The result supports the narrower conclusion that class selection changes queue waiting time under the specified shared backlog.

5) Video-Class Interpretation: Video packets are five times the voice payload size in the baseline, and their offered rate is 1.92 Mb/s. FIFO video delay is close to voice delay because both classes share the same arrival-order queue. Strict priority reduces video mean delay to 3.319 ms and its 95th percentile to 5.180 ms. WDRR records a mean of 19.143 ms and a 95th percentile of 32.522 ms. Compared with strict priority, WDRR accepts higher video delay in exchange for a less extreme delay for the lowest class, while still keeping video delay far below FIFO in this model.

Interactive and buffered video have different tolerance to delay and variation. The model has one video class and no playback buffer, rate adaptation, frame deadlines, or frame dependencies. The figures therefore describe packet service for the chosen synthetic stream, not perceptual video quality.

6) Best-Effort Interpretation: Best-effort traffic carries the largest offered rate, 3.84 Mb/s, and packets of 1200 bytes. FIFO mean delay is 304.000 ms. Strict priority increases it to 671.373 ms, while WDRR records 645.874 ms. WDRR's mean is 25.499 ms, or about 3.80%, lower than strict priority's value in this baseline. Both remain much higher than FIFO because the common system is overloaded and the higher classes are served ahead of, or receive larger configured quantum than, best effort.

The buffer experiment makes the risk clearer. At a 500-packet buffer, best-effort mean delay rises to 1653.802 ms under strict priority and 1628.039 ms under WDRR. The buffer holds more work, but a packet near the back of a long queue waits longer. WDRR continues to serve the class, but a positive long-run service share does not guarantee a short queueing delay when the class's demand and total offered load remain high.

7) Loss and Payload Goodput Across Classes: At the overloaded baseline, the nine class-policy loss values span 26.210% to 26.629%, a range of 0.419 percentage points. This small range is consistent with a shared drop-tail buffer that has no class-specific admission protection. The low-loss configuration at 6.5 Mb/s and 50 packets is different: there, drops arise from brief bursts, and the total count is 560 across the displayed policy results. Both regimes need to be stated when discussing loss.

Baseline class goodputs sum to approximately 4.5 Mb/s for every policy: 4.500 Mb/s for FIFO and strict priority, and 4.499 Mb/s for WDRR after rounding. The per-class values are also close to the fraction of offered payload that remains after drops. Since the three classes have different demand, equal absolute goodput would not be a suitable fairness target. A more interpretable supplemental quantity is delivered goodput

TABLE V: Baseline performance averaged over ten random seeds ($C = 4.5$ Mb/s, buffer size = 200 packets).

Policy	Class	Mean delay (ms)	Delay std. (ms)	95th-percentile delay (ms)	Packet loss (%)	Goodput (Mb/s)
FIFO	Best effort	304.000	0.946	324.667	26.629	2.803
FIFO	Video	303.794	1.038	324.353	26.416	1.405
FIFO	Voice	302.366	1.017	322.935	26.501	0.292
Strict priority	Best effort	671.373	2.252	724.467	26.458	2.804
Strict priority	Video	3.319	0.014	5.180	26.355	1.403
Strict priority	Voice	1.458	0.005	2.381	26.210	0.293
WDRR	Best effort	645.874	1.219	698.031	26.437	2.805
WDRR	Video	19.143	0.286	32.522	26.394	1.402
WDRR	Voice	23.411	0.313	45.075	26.407	0.292

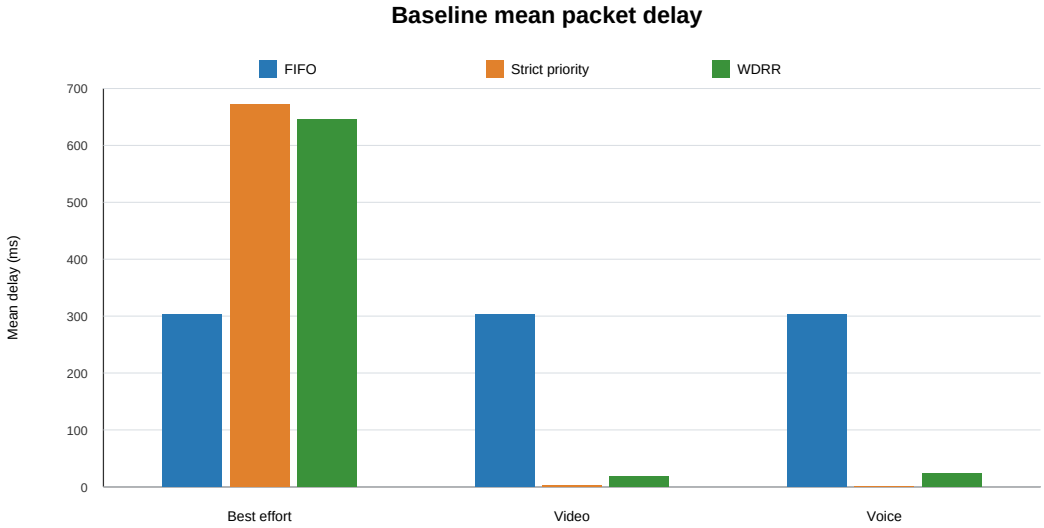


Fig. 2: Baseline mean delay by traffic class and scheduling policy. The large best-effort delay under strict priority and WDRR is shown on the same scale as the low voice and video delays.

TABLE VI: Approximate 95% t intervals for baseline mean delay across ten seeds.

Policy and class	Mean (ms)	SD (ms)	Lower (ms)	Upper (ms)
FIFO, best effort	304.000	0.946	303.323	304.677
FIFO, video	303.794	1.038	303.051	304.537
FIFO, voice	302.366	1.017	301.639	303.094
Strict priority, best effort	671.373	2.252	669.763	672.983
Strict priority, video	3.319	0.014	3.309	3.329
Strict priority, voice	1.458	0.005	1.454	1.462
WDRR, best effort	645.874	1.219	645.002	646.746
WDRR, video	19.143	0.286	18.938	19.348
WDRR, voice	23.411	0.313	23.187	23.635

divided by offered goodput for that class; in the baseline table, these ratios are all approximately 0.73. This indicates that the similar loss values correspond to similar proportional delivery in this particular scenario.

8) Packet Loss and Goodput: Packet-loss ratios are similar across policies, ranging from 26.210% to 26.629% in the baseline case. The aggregate workload exceeds the fixed link capacity, so changing packet order does not remove the underlying overload. In this experiment, scheduling has a clearer effect on the distribution of delay among classes than on the overall loss ratio.

The measured class goodputs are also similar across policies. Their sums are approximately 4.5 Mb/s, consistent with the configured bottleneck capacity. The small differences between policies arise from the finite simulation and traffic realization. Overall, the baseline results indicate that WDRR redistributes delay among the traffic classes; it does not create additional link capacity or eliminate congestion.

G. Effect of Link Capacity

Table VIII reports the class-specific mean delay when the link capacity is varied and the buffer is fixed at 200 packets. The

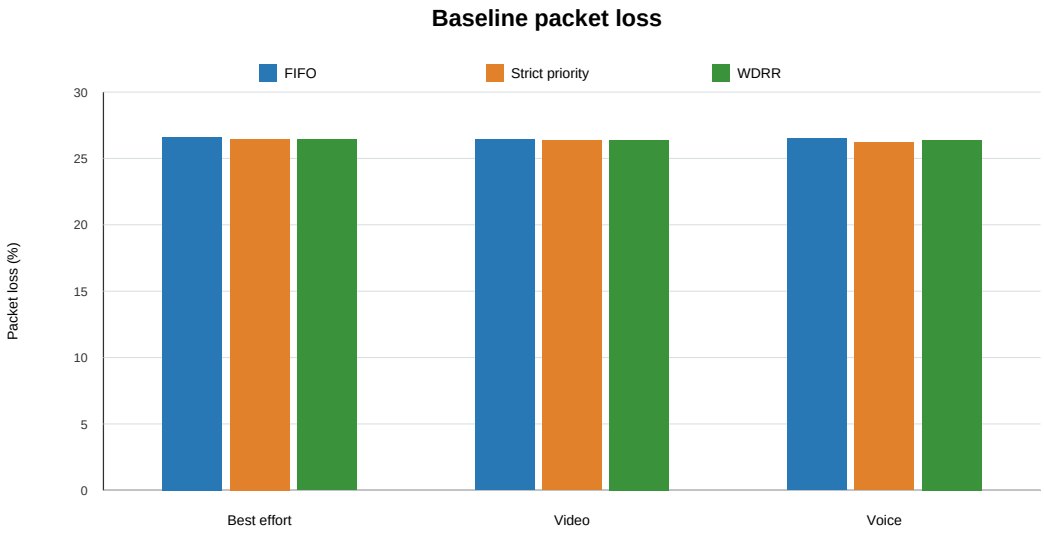


Fig. 3: Baseline packet-loss ratios. Loss is similar across schedulers because the shared finite buffer applies the same drop-tail rule to every class.

TABLE VII: Baseline mean-delay change relative to FIFO.

Policy	Voice	Video	Best effort
Strict priority	-99.52%	-98.91%	$+120.85\%$
WDRR	-92.26%	-93.70%	$+112.46\%$

offered traffic remains $6.16\ \mathrm{Mb/s}$ in all cases.

At $4.5\ \mathrm{Mb/s}$, the offered load exceeds capacity by a substantial margin. Increasing capacity to $5.0\ \mathrm{Mb/s}$ reduces the mean delays, but the link remains overloaded because the offered rate is still $6.16\ \mathrm{Mb/s}$. Class-specific packet-loss values at these two capacities are approximately 26.2–26.6% and 18.1–18.5%, respectively.

At $6.5\ \mathrm{Mb/s}$, the link capacity exceeds the aggregate offered rate. With buffer capacities of 200 and 500 packets, no packets are dropped. With the 50-packet buffer, brief arrival bursts cause a small number of drops: the class-specific loss ratios range from 0.055% to 0.060% under FIFO, from 0.017% to 0.021% under strict priority, and from 0.021% to 0.033% under WDRR. Thus, a small finite buffer can overflow temporarily even when the average link capacity exceeds the average offered traffic. Goodputs approach the offered rates, and packet delays are substantially lower than in the overloaded cases. For the 200-packet buffer used in Table VIII, no packet loss was observed at $6.5\ \mathrm{Mb/s}$.

The scheduling policies continue to distribute delay differently. Strict priority gives the lowest voice and video delays, while its best-effort delay remains higher. WDRR produces intermediate delays for the three classes. These results show that additional link capacity reduces congestion and loss, whereas the scheduling policy determines how the remaining delay is distributed among traffic classes.

H. Effect of Buffer Capacity

Table IX compares the scheduling policies for buffer capacities of 50, 200, and 500 packets while the link capacity is fixed at $4.5\ \mathrm{Mb/s}$.

Increasing the buffer capacity reduces packet loss slightly, but it substantially increases delay when the offered load exceeds the link capacity. For example, the WDRR mean delay for best-effort traffic increases from $140.739\ \mathrm{ms}$ with a 50-packet buffer to $645.874\ \mathrm{ms}$ with a 200-packet buffer and $1628.039\ \mathrm{ms}$ with a 500-packet buffer. A larger buffer can hold more packets during overload, but those packets may wait longer before transmission.

The effect depends on the scheduling policy and traffic class. Under strict priority, voice and video delays remain low as the buffer grows, while the best-effort delay increases sharply. WDRR also maintains much lower voice and video delays than FIFO, while its best-effort delay remains high under sustained overload. The class-specific packet-loss ratios decrease only modestly as the buffer grows, from approximately 26.6–26.9% at 50 packets to 25.5–26.0% at 500 packets.

These results show that a larger buffer does not eliminate congestion. It trades a small reduction in packet loss for longer queueing delays. Consequently, buffer capacity should be considered together with link capacity and the scheduling policy.

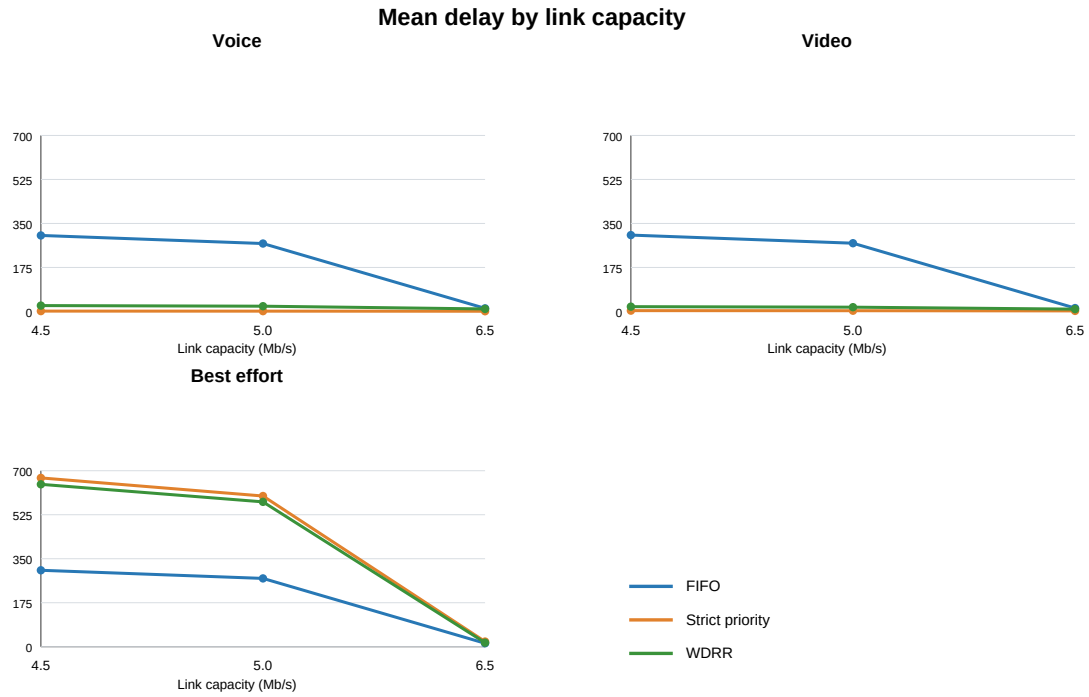


Fig. 4: Class-specific mean delay as link capacity changes with the buffer fixed at 200 packets.

TABLE VIII: Mean packet delay (ms) versus link capacity with a 200-packet buffer.

Capacity (Mb/s)	Policy	Voice (ms)	Video (ms)	Best effort (ms)
4.5	FIFO	302.366	303.794	304.000
4.5	Strict priority	1.458	3.319	671.373
4.5	WDRR	23.411	19.143	645.874
5.0	FIFO	270.213	271.492	271.661
5.0	Strict priority	1.283	3.006	599.373
5.0	WDRR	20.981	17.092	576.111
6.5	FIFO	12.451	13.510	13.701
6.5	Strict priority	0.912	2.264	20.518
6.5	WDRR	10.082	9.307	16.045

I. Discussion and Model Limitations

The experiments reveal three complementary aspects of the scheduling problem. First, the scheduler determines how queueing delay is distributed among traffic classes. FIFO treats packets uniformly, which results in similar class delays but does not protect voice and video during overload. Strict priority protects these delay-sensitive classes, but it can leave best-effort packets waiting for substantially longer. WDRR provides an intermediate outcome under the selected weight ratio: voice and video receive lower delays than under FIFO, while best-effort delay is somewhat lower than under strict priority.

Second, scheduling cannot compensate for insufficient link capacity. When the offered rate exceeds the service rate, a finite buffer eventually fills and packets are dropped. Raising the link capacity from 4.5 Mb/s to 5.0 Mb/s reduces congestion, but the offered load remains greater than capacity. At 6.5 Mb/s, capacity exceeds the offered load and the simulation records no packet loss. This distinction separates the effect of the scheduler, which redistributes service, from the effect of capacity, which changes the amount of available service.

Third, increasing buffer size primarily changes the delay–loss trade-off. A larger buffer can retain more packets during overload and slightly reduce packet loss, but it also permits longer queues to form. The best-effort results show that this additional buffering can lead to very large delays under strict priority and WDRR. Buffer capacity should therefore be selected with delay requirements in mind, rather than optimized solely to minimize packet drops.

The experiment uses paired traffic traces across policies and ten random-seed replications for each configuration. This supports a consistent descriptive comparison. The reported standard deviations summarize variation across seeds; they are not, by themselves, evidence of statistical significance. No significance test is performed in this study.

The model is intentionally limited to queueing and packet scheduling at one access-point bottleneck. It uses independent

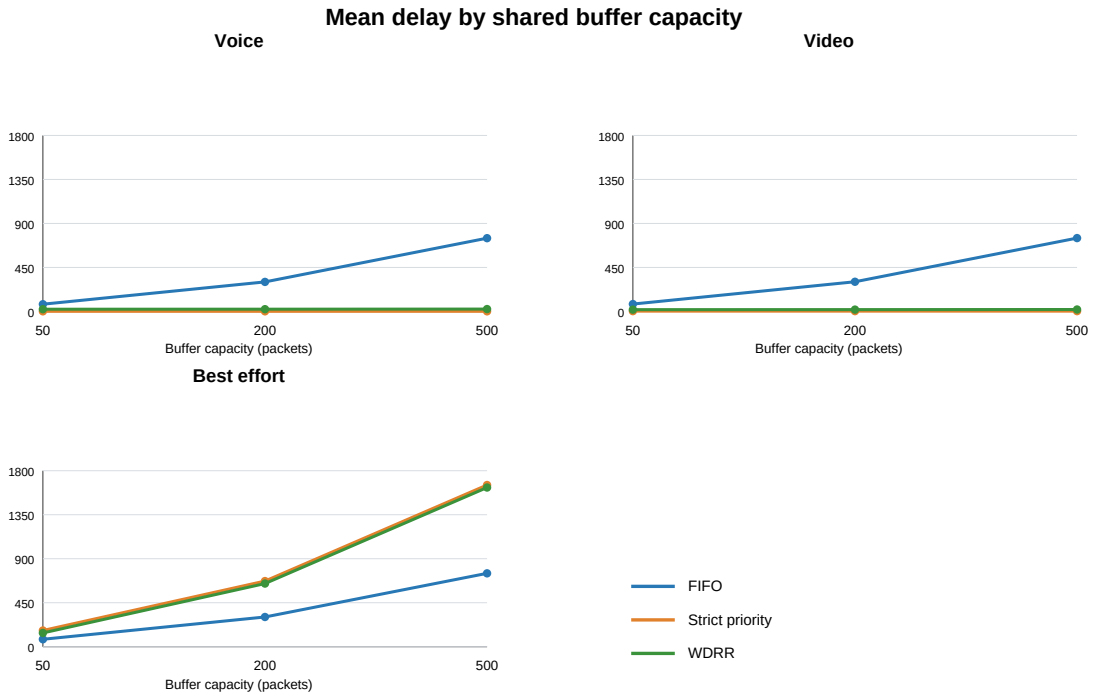


Fig. 5: Class-specific mean delay as the shared buffer changes at $C = 4.5$ Mb/s.

TABLE IX: Mean packet delay (ms) versus buffer capacity at $C = 4.5$ Mb/s.

Buffer (packets)	Policy	Voice (ms)	Video (ms)	Best effort (ms)
50	FIFO	74.317	75.765	76.068
50	Strict priority	1.459	3.309	165.956
50	WDRR	23.130	18.675	140.739
200	FIFO	302.366	303.794	304.000
200	Strict priority	1.458	3.319	671.373
200	WDRR	23.411	19.143	645.874
500	FIFO	749.522	750.444	750.442
500	Strict priority	1.454	3.338	1653.802
500	WDRR	24.556	20.123	1628.039

Poisson arrivals, fixed payload sizes, constant link capacity, and a finite packet buffer. It does not model wireless propagation, collisions, retransmissions, rate adaptation, or the distributed channel contention of IEEE 802.11 EDCA. Consequently, the results should be interpreted as outcomes of the specified queueing model, not as guarantees for a deployed Wi-Fi network.

V. Conclusions

This paper compared FIFO, strict-priority scheduling, and a Weighted Deficit Round Robin (WDRR) scheduler for voice, video, and best-effort traffic at a shared access-point bottleneck. The evaluation used a reproducible discrete-event simulation with paired arrival traces, ten random seeds per configuration, three link capacities, and three buffer limits.

Under the overloaded baseline ($C = 4.5$ Mb/s and a 200-packet buffer), FIFO produced similar mean delays across classes, between approximately 302 and 304 ms. Strict priority reduced voice and video mean delay to 1.458 ms and 3.319 ms, respectively, but increased best-effort mean delay to 671.373 ms. WDRR provided a different service trade-off: voice and video mean delays were 23.411 ms and 19.143 ms, while best-effort mean delay was 645.874 ms. Packet-loss ratios remained close across policies, at about 26.2–26.6%, because the offered load exceeded the bottleneck capacity.

The sensitivity analysis showed that increasing link capacity reduced delay and loss; at 6.5 Mb/s, the capacity exceeded the aggregate offered load of 6.16 Mb/s. Increasing buffer size reduced loss modestly under overload but caused longer queueing delays, particularly for best-effort traffic. These results show that scheduling redistributes service among classes, while link capacity and buffer size determine the broader congestion and delay-loss trade-off.

The study is limited to a single access-point queueing bottleneck with independent Poisson arrivals, fixed packet sizes, constant service capacity, and a finite buffer. It does not model radio propagation, collisions, retransmissions, rate adaptation, or distributed IEEE 802.11 EDCA contention. The

findings therefore characterize the specified simulation rather than predict performance in a deployed WLAN. Future work could extend the model with variable wireless link rates and EDCA contention, and examine additional WDRR weight configurations and confidence intervals.

A. Recommended Follow-Up Experiments

A next-stage evaluation can extend the model in manageable steps while preserving the baseline as a control. Table X lists a sequence of experiments, each with a measurable objective. The first step should reproduce the current output from the exact multi-seed source notebook and archive the run-level CSVs. Later steps can add one new mechanism at a time so its effect remains interpretable.

Each stage retains the same discipline used in the present study: specify the model boundary, record every configuration, pair traces where policies can be compared on the same input, and report class-level metrics beside aggregate values. This roadmap turns the queueing model into a baseline for a broader WLAN evaluation without treating the current results as an 802.11 performance guarantee.

Appendix A
Reproduction Configuration

This appendix records the configuration needed to understand the experiments and to reconstruct their intended workload. The model uses three independent traffic sources, a common access-point bottleneck, a shared waiting buffer, and the same packet-arrival trace across policies within each paired replication.

The run-level factors are summarized in Table XII. The baseline is the 4.5-Mb/s, 200-packet-buffer case. Capacity sensitivity changes capacity while holding the buffer at 200 packets. Buffer sensitivity changes the buffer while holding capacity at 4.5 Mb/s. These are one-factor-at-a-time comparisons around the baseline; they do not test every combination of capacity and buffer size.

Three capacities multiplied by three buffer values, three policies, and ten replications give 270 policy runs when all nine capacity-buffer scenarios are included. The paired-arrival design means that the three policies for a given seed and scenario share the same trace. It does not mean that the trace for one seed is identical to the trace for another seed.

A. Reproduction Workflow

A complete reproduction should preserve the exact source notebook or script, the seed list, the traffic configuration, and the exported run-level records. The recommended workflow is to run all cells from a clean notebook state, save per-seed results before calculating summary tables, and use the same aggregation rules for every policy. If plots are created from a table, the plot should be regenerated after any source data change rather than edited manually.

For each run, the source interval ends at 60 s. The queue is then drained until every admitted packet has completed service. Delay is calculated using each packet's arrival and completion timestamps. Goodput uses the 60-s source interval as its denominator, not the longer drain interval. A packet that is dropped contributes to the loss numerator and has no delivery delay or goodput contribution.

The source listing in Appendix E is the three-policy Colab baseline script supplied with the project. It runs one seed at the baseline capacity and buffer. The ten-seed averages and the capacity/buffer sensitivity tables are the separately supplied extended experiment results. The one-seed listing is included to make the baseline model inspectable; it must not be presented as a stand-alone reproducer of every aggregated table.

Appendix B
Metric Calculations and Scheduler Details

A. Offered Rate and Load Ratio

For packet rate λ_i packets/s and payload size L_i bytes, the offered rate in megabits per second is

$$R_i = \frac{8\lambda_i L_i}{10^6}. \tag{21}$$

The baseline calculations are $8(250)(200)/10^6 = 0.40$ Mb/s for voice, $8(240)(1000)/10^6 = 1.92$ Mb/s for video, and $8(400)(1200)/10^6 = 3.84$ Mb/s for best effort. The sum is 6.16 Mb/s. Dividing by capacity gives $\rho = 1.369$ at 4.5 Mb/s, $\rho = 1.232$ at 5.0 Mb/s, and $\rho = 0.948$ at 6.5 Mb/s.

The load ratio compares mean offered payload to nominal service capacity. When it is above one, the finite-buffer model cannot serve the long-run mean offered load, so drops are expected after transient effects. When it is below one, the average load is sustainable in the idealized constant-rate model, but bursts can still temporarily exceed the waiting-buffer limit. The 560 drops in the 6.5-Mb/s, 50-packet configuration are an example of this short-term overflow; they do not contradict the average load ratio below one.

B. Loss and Goodput Accounting

For class i, the loss percentage is $100N_i^{\mathrm{drop}}/N_i^{\mathrm{gen}}$. Delivered goodput is $8B_i^{\mathrm{del}}/(T_{\mathrm{gen}}10^6)$ Mb/s, where B_i^{del} is delivered payload bytes. The total goodput check sums these class payload rates and verifies that they do not exceed the modeled link capacity. The check passed for the reported experiment runs.

In an overloaded run, the sum of class goodputs is approximately the bottleneck capacity, subject to packet-boundary effects and the finite run. This is not PHY throughput: no MAC headers, interframe spaces, acknowledgments, retransmissions, or contention overhead are counted. At 6.5 Mb/s, class goodputs approach the offered rates because average capacity exceeds average demand, except that finite-buffer loss can reduce a small amount of delivered payload in the 50-packet case.

C. WDRR Quantum Interpretation

With base quantum $q_0 = 1500$ bytes and weights $(1, 5, 5)$, the quanta are 1500 bytes for voice and 7500 bytes each for video and best effort. If all three queues are continuously backlogged, the nominal shares of 4.5 Mb/s are approximately 0.409, 2.045,

TABLE X: Planned extensions for a protocol-level QoS study.

Stage	Extension	Main measurements
1. Reproduction	Archive the exact extended notebook, seed list, and per-run outputs	Recomputed tables, consistency checks, confidence intervals
2. Traffic traces	Replace independent Poisson sources with measured or bursty traces	Delay percentiles, burst loss, class occupancy
3. Class buffers	Compare shared and per-class queue limits and drop policies	Per-class loss, delay, and buffer utilization
4. Wireless MAC	Implement one AP and stations with QoS-enabled EDCA in ns-3	Airtime, retries, contention delay, application goodput
5. Sensitivity	Vary station count, PHY rate, and EDCA parameters one at a time	Robustness of class differentiation under contention
6. Statistical depth	Increase independent seeds and report intervals for every metric	Stability of means, tail estimates, and policy contrasts

TABLE XI: Complete baseline workload and experiment factors.

Item	Voice	Video	Best effort
Arrival process	Poisson	Poisson	Poisson
Mean arrival rate	250 packets/s	240 packets/s	400 packets/s
Fixed payload	200 bytes	1000 bytes	1200 bytes
Mean offered rate	0.40 Mb/s	1.92 Mb/s	3.84 Mb/s
WDRR weight	1	5	5
Strict-priority rank	1 (highest)	2	3 (lowest)

TABLE XII: Run matrix and fixed settings.

Factor	Values
Source generation interval	60 s
Random seeds	2026–2035
Replications per setting	10
Policies	FIFO, strict priority, WDRR
Baseline link capacity	4.5 Mb/s
Capacity sweep	4.5, 5.0, 6.5 Mb/s
Baseline waiting buffer	200 packets
Buffer sweep	50, 200, 500 packets
WDRR weight ratio	1:5:5
Service rule	Non-preemptive, payload-rate model

and 2.045 Mb/s. If a class has no packets, its configured share is not wasted: the scheduler skips empty queues and the active classes can use the available server. Consequently, measured class goodput depends on offered load as well as weights.

The configured ratio gives voice one fifth of the quantum assigned to either of the other classes. It does not mean that voice receives one fifth of the packets, nor that its delay is exactly five times another class's delay. Packet sizes differ, arrivals are stochastic, and service is non-preemptive. The deficit counter accounts for bytes, while the queueing delay depends on backlog and visit timing.

Appendix C
Low-Loss Validation at 6.5 Mb/s

Table XIII reports the class-specific drop counts supplied for the 6.5-Mb/s case with a 50-packet shared buffer. There were 560 drops in total. Relative to the generated packet counts, losses are below 0.061% for every class and policy. With 200- and 500-packet buffers, the supplied checks reported zero drops at this capacity.

The generated counts repeat across policies because each scheduler receives the same trace; 533743 packets were generated per policy over the ten seeds. The total of 560 drops is summed across the three policy runs and is not a single-policy loss rate. The data show that brief arrival bursts can fill a small buffer even when mean offered load is below link capacity. They do not establish a general packet-loss advantage for any scheduler: the number of drops is small, depends on the trace and service order, and the overloaded baseline exhibits much larger and more similar loss ratios.

Appendix D
Interpretation Guide and Glossary

The following terms are used in the report with the meanings listed in Table XIV. This helps distinguish queue-model outputs from broader wireless networking metrics.

A reader should interpret the numeric results in this order: first identify the capacity and buffer regime; next compare class-specific delay and loss; then inspect goodput to see whether delivered service changed; finally consider whether the result depends on the queueing abstraction. This order avoids treating low delay among delivered packets as a complete performance claim when dropped packets are excluded from delay calculations.

Appendix E
Colab Baseline Source Code

The following listing is the baseline Python code supplied for the project. It generates one shared Poisson arrival trace at seed 2026, runs FIFO, strict priority, and WDRR with the baseline capacity and buffer, prints class metrics, plots mean delay and packet loss, and saves the one-seed output. It is presented as an inspectable baseline implementation. The extended ten-seed and sensitivity aggregates in the report require the extended notebook runs described in Appendix A.

Listing 1: Supplied single-seed baseline simulation listing.

```
import heapq
```

TABLE XIII: Packet drops at $C = 6.5$ Mb/s with a 50-packet shared buffer.

Policy	Class	Generated	Dropped	Loss (%)
FIFO	Best effort	239986	144	0.06000
FIFO	Video	143881	86	0.05977
FIFO	Voice	149876	83	0.05538
Strict priority	Best effort	239986	42	0.01750
Strict priority	Video	143881	25	0.01738
Strict priority	Voice	149876	31	0.02068
WDRR	Best effort	239986	71	0.02959
WDRR	Video	143881	47	0.03267
WDRR	Voice	149876	31	0.02068
Total	Three policies	–	560	–

TABLE XIV: Selected terms and symbols.

Term	Meaning in this report
AP	Wireless access point that classifies the modeled incoming traffic
Best effort	The lowest-priority data class in the simulation
Buffer capacity	Maximum number of waiting packets; packet in service is not counted
EDCA	IEEE 802.11 contention-based QoS access method; not implemented in the simulator
Goodput	Successfully delivered payload bits divided by the source-generation duration
Mean delay	Average arrival-to-completion time over delivered packets
P95 delay	95th percentile of delivered packet delays, calculated per run then averaged
Packet loss	Fraction of generated packets dropped at the finite waiting buffer
QoS	Quality of service; differentiated treatment based on application/network needs
λ_i	Mean packet arrival rate of class i in packets per second
L_i	Fixed payload size for class i in bytes
C	Constant modeled payload service rate in bits per second
ρ	Mean offered payload rate divided by the modeled service capacity
w_i	Configured WDRR weight of class i
q_i	Byte quantum assigned to class i during a WDRR visit

```python
import random
import pandas as pd
import matplotlib.pyplot as plt

# Baseline configuration
SIMULATION_TIME = 60.0
CAPACITY_MBPS = 4.5
BUFFER_PACKETS = 200
SEED = 2026

CLASSES = {
    "Voice":      {"arrival_rate": 250, "size_bytes": 200,
      "weight": 1},
    "Video":      {"arrival_rate": 240, "size_bytes": 1000,
      "weight": 5},
    "Best effort":{"arrival_rate": 400, "size_bytes": 1200,
      "weight": 5},
}

POLICIES = ["FIFO", "Strict Priority", "WDRR"]


def generate_arrivals(seed):
    """Generate one shared Poisson arrival trace for all
     policies."""
    rng = random.Random(seed)
    events = []

    for class_name, cfg in CLASSES.items():
        t = 0.0
        while True:
            t += rng.expovariate(cfg["arrival_rate"])
            if t >= SIMULATION_TIME:
                break
            events.append((t, class_name, cfg["size_bytes"
     ]))

    events.sort(key=lambda x: x[0])
    return events


def simulate(policy, arrivals):
    queues = {name: [] for name in CLASSES}
    results = {
        name: {"generated": 0, "dropped": 0, "delays": [],
      "bytes": 0}
        for name in CLASSES
    }

    # Event queue: (completion_time, class_name,
     packet_arrival, packet_size)
    completions = []
    busy = False
    queued_packets = 0
    arrival_index = 0
    now = 0.0

    # WDRR state
    weights = {name: cfg["weight"] for name, cfg in CLASSES.
     items()}
    quantum_bytes = {name: 1500 * weights[name] for name in
      CLASSES}
    deficit = {name: 0 for name in CLASSES}
    wdrr_order = ["Voice", "Video", "Best effort"]
    wdrr_cursor = 0

    def choose_next():
        nonlocal wdrr_cursor

        if policy == "FIFO":
            candidates = [
                (q[0][0], name)
                for name, q in queues.items()
                if q
            ]
            if not candidates:
                return None
            _, selected = min(candidates)
            return selected

        if policy == "Strict Priority":
            for name in ["Voice", "Video", "Best effort"]:
                if queues[name]:
                    return name
            return None

        # Weighted Deficit Round Robin
        if not any(queues.values()):
```

```
            return None

        checked_empty = 0
        while True:
            name = wdrr_order[wdrr_cursor]

            if not queues[name]:
                deficit[name] = 0
                wdrr_cursor = (wdrr_cursor + 1) % len(wdrr_order)
                checked_empty += 1
                if checked_empty >= len(wdrr_order):
                    return None
                continue

            checked_empty = 0
            packet_size = queues[name][0][1]

            if deficit[name] < packet_size:
                deficit[name] += quantum_bytes[name]
                wdrr_cursor = (wdrr_cursor + 1) % len(wdrr_order)
                continue

            deficit[name] -= packet_size
            return name

    def start_next(current_time):
        nonlocal busy, queued_packets

        selected = choose_next()
        if selected is None:
            busy = False
            return

        packet_arrival, packet_size = queues[selected].pop(0)
        queued_packets -= 1

        service_time = (packet_size * 8) / (CAPACITY_MBPS * 1_000_000)
        finish_time = current_time + service_time
        heapq.heappush(
            completions,
            (finish_time, selected, packet_arrival, packet_size)
        )
        busy = True

    while arrival_index < len(arrivals) or completions:
        next_arrival_time = (
            arrivals[arrival_index][0]
            if arrival_index < len(arrivals)
            else float("inf")
        )
        next_completion_time = (
            completions[0][0] if completions else float("inf")
        )

        if next_arrival_time <= next_completion_time:
            now, class_name, packet_size = arrivals[arrival_index]
            arrival_index += 1
            results[class_name]["generated"] += 1

            if queued_packets >= BUFFER_PACKETS:
                results[class_name]["dropped"] += 1
                continue

            queues[class_name].append((now, packet_size))
            queued_packets += 1

            if not busy:
                start_next(now)

        else:
            now, class_name, packet_arrival, packet_size = heapq.heappop(completions)
            results[class_name]["delays"].append(now - packet_arrival)
            results[class_name]["bytes"] += packet_size
            busy = False
            start_next(now)

    rows = []
    for class_name, values in results.items():
        delays = values["delays"]
        generated = values["generated"]
        delivered = len(delays)

        rows.append({
            "Policy": policy,
            "Class": class_name,
            "Mean delay (ms)": (
                1000 * sum(delays) / delivered if delivered else 0
            ),
            "95th percentile delay (ms)": (
                1000 * float(pd.Series(delays).quantile(0.95))
                if delivered else 0
            ),
            "Packet loss (%)": (
                100 * values["dropped"] / generated if generated else 0
            ),
            "Goodput (Mb/s)": (
                8 * values["bytes"] / SIMULATION_TIME / 1_000_000
            ),
            "Generated": generated,
            "Delivered": delivered,
            "Dropped": values["dropped"],
        })

    return rows


# Use the same traffic arrivals for all three scheduling policies.
arrival_trace = generate_arrivals(SEED)
all_results = []

for policy in POLICIES:
    all_results.extend(simulate(policy, arrival_trace))

df = pd.DataFrame(all_results)
print(df.round(3).to_string(index=False))

# Plot class-specific mean delay and packet loss.
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for metric, axis, title, ylabel in [
    ("Mean delay (ms)", axes[0], "Mean packet delay", "Delay (ms)"),
    ("Packet loss (%)", axes[1], "Packet loss", "Loss (%)"),
]:
    pivot = df.pivot(index="Class", columns="Policy", values=metric)
    pivot.plot(kind="bar", ax=axis)
    axis.set_title(title)
    axis.set_ylabel(ylabel)
    axis.set_xlabel("")
    axis.grid(axis="y", linestyle="--", alpha=0.4)
    axis.legend(title="Scheduler")

plt.tight_layout()
plt.savefig("qos_baseline_results.png", dpi=300, bbox_inches="tight")
plt.show()

# Save the numerical results for later use in the report.
df.to_csv("qos_baseline_results.csv", index=False)
```

APPENDIX F
THREATS TO VALIDITY AND INTERPRETATION

A. Construct Validity

Construct validity concerns whether a metric represents the concept named in the report. The simulator records arrival-to-completion time, so “delay” includes queue waiting and serialization at one bottleneck. It does not include application processing, wired-path delay, radio contention, or retransmission. Packet loss means drop-tail buffer overflow only. Goodput

counts payload bytes and excludes headers and MAC overhead. These definitions are explicit, but they are narrower than end-to-end latency, Wi-Fi loss, or link throughput in a real deployment. The three traffic labels are also simplified constructs. Voice, video, and best effort stand for traffic classes, not measured codecs or application traces. A single video stream cannot represent both a conversational call and an adaptive buffered stream. The fixed packet-size assumptions make the comparison tractable but remove within-class variation. Results should therefore be stated as outcomes for this synthetic workload.

B. Internal Validity

Internal validity asks whether policy changes, rather than a hidden configuration change, explain the observed differences. The experiment uses paired arrival traces, a fixed capacity within each scenario, a common buffer limit, and the same traffic-generation interval. These choices support a controlled comparison. The packet-accounting check verifies that generated packets are partitioned into delivered and dropped packets. The goodput-capacity check helps detect an impossible aggregate service result.

A remaining implementation risk is event ordering when an arrival timestamp coincides exactly with a service completion. The event loop defines a tie rule, and that rule should be kept identical across policies. The source code should also be retained with the exported data so a reviewer can verify the queue-count convention, WDRR deficit reset, and service-completion timing. These details can affect edge cases even when the model equations are unchanged.

C. External Validity

External validity concerns generalization beyond the modeled system. The experiment uses one access point, one constant-rate server, a shared buffer, independent Poisson sources, and fixed payload sizes. It omits station-level contention, hidden terminals, channel errors, rate adaptation, aggregation, retransmissions, power-save behavior, and mobility. The wireless channel is therefore represented by a fixed service-rate abstraction. Findings do not predict the exact delay, loss, or throughput of an IEEE 802.11 deployment.

The experiment also samples only three link capacities, three buffer sizes, one WDRR weight ratio, and one class-priority order. Other parameter combinations may produce different trade-offs. The values support a comparison of the specified cases; they do not identify a globally optimal buffer size or scheduler configuration.

D. Statistical Validity

Ten random seeds provide more information than a single run, but the sample remains limited. The reported standard deviations summarize the variability of the run means, and the approximate intervals in Section IV-F1 rely on the usual independent-run interpretation. No statistical significance test is performed. Sensitivity tables provide means without seed-level dispersion because those dispersion values were not part of the supplied output. Accordingly, differences of a few milliseconds in those tables should be treated as descriptive, not as statistically established.

The 95th-percentile delay is computed inside each run and then averaged over seeds. This is not the same as pooling all packets from all seeds and taking one percentile. The two procedures answer different questions. The run-level method gives each seed equal influence; a pooled percentile gives more influence to runs with more delivered packets. The report uses the former consistently for the baseline summary.

TABLE XV: Best-effort mean-delay increase as the buffer grows.

Policy	200/50 packets	500/50 packets	500/200 packets
FIFO	4.00×	9.86×	2.47×
Strict priority	4.05×	9.97×	2.46×
WDRR	4.59×	11.57×	2.52×

Appendix G
Extended Numerical Comparison

The baseline and sensitivity tables in the main text give the primary values. Table XV shows how best-effort mean delay scales between the tested buffer capacities. Multipliers are ratios of the reported means, not additional simulation measurements.

The values reinforce the overload interpretation. Increasing the buffer from 50 to 500 packets reduces loss only modestly, while the best-effort mean delay rises by roughly an order of magnitude for each policy. The high-priority classes behave differently: strict-priority voice and video mean delays remain almost constant across the tested buffer limits, and WDRR voice/video means change only slightly. The policy determines which class accumulates the additional waiting time.

The link-capacity experiment supplies a complementary result. At 5.0 Mb/s, the average offered load still exceeds service capacity, so the system remains overloaded even though delay and loss improve relative to the 4.5-Mb/s baseline. At 6.5 Mb/s, the mean offered rate is below capacity, and the 200-packet baseline buffer records no packet drops in the supplied checks. The 50-packet case still experiences brief burst overflow. Capacity changes the amount of service available; buffer size changes how much transient work can wait; the scheduler changes how that service is distributed.

These statements separate three effects that can otherwise be conflated. A delay improvement after raising capacity is not evidence that a scheduler became fairer. A loss decrease after enlarging the buffer does not mean congestion disappeared. A lower high-class delay under strict priority does not mean aggregate goodput increased. Each conclusion should be tied to the variable that was changed and to the metric that was measured.

Appendix H
Data and Reproduction Checklist

Before presenting or reusing the results, verify the following items against the archived notebook and CSV outputs:

1) Confirm the class rates, payload sizes, generation interval, and offered-rate calculations.

2) Confirm that the seed list and number of replications match the report configuration.
3) Confirm that all policies use the same arrival trace for each seed and scenario.
4) Confirm whether the buffer limit counts waiting packets only or includes the packet in service; use the same convention for every policy.
5) Verify the accounting identity $N^{\mathrm{gen}} = N^{\mathrm{del}} + N^{\mathrm{drop}}$ for every run and class.
6) Confirm that delay is recorded at service completion and that the post-generation drain is included.
7) Confirm that goodput uses the 60-second generation interval as its denominator.
8) Recompute each table from the run-level data and retain the unrounded values for any new analysis.
9) Regenerate all figures from the numerical tables after any data or label change.
10) Keep the source, output tables, figures, and final PDF together as one versioned project bundle.

This checklist is not a replacement for source review. It is a compact audit trail for the assumptions most likely to change a result or its interpretation. If any item differs in a later notebook version, the configuration and affected tables should be updated together rather than silently mixing output from different runs.

REFERENCES

[1] IEEE, "Ieee standard for information technology—telecommunications and information exchange between systems—local and metropolitan area networks—specific requirements—part 11: Wireless lan medium access control (mac) and physical layer (phy) specifications," IEEE, Tech. Rep. IEEE Std 802.11-2024, 2025. [Online]. Available: https://standards.ieee.org/ieee/802.11/10548/

[2] T. Szigeti, J. Henry, and F. Baker, "Mapping diffserv to ieee 802.11," Internet Engineering Task Force, RFC 8325, 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8325.html

[3] ITU-T, "End-user multimedia qos categories," International Telecommunication Union, Recommendation G.1010, 2001. [Online]. Available: https://www.itu.int/rec/T-REC-G.1010-200111-I/en

[4] J. Babiarz, K. Chan, and F. Baker, "Configuration guidelines for diffserv service classes," Internet Engineering Task Force, RFC 4594, 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4594.html

[5] K. Nichols, S. Blake, F. Baker, and D. Black, "Definition of the differentiated services field in the ipv4 and ipv6 headers," Internet Engineering Task Force, RFC 2474, 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2474.html

[6] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "An architecture for differentiated services," Internet Engineering Task Force, RFC 2475, 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2475.html

[7] IEEE, "Ieee standard for information technology—telecommunications and information exchange between systems—specific requirements—part 11: Wireless lan medium access control and physical layer specifications—amendment 8: Medium access control quality of service enhancements," IEEE, Tech. Rep. IEEE Std 802.11e-2005, 2005.

[8] M. Shreedhar and G. Varghese, "Efficient fair queuing using deficit round-robin," *IEEE/ACM Transactions on Networking*, vol. 4, no. 3, pp. 375–385, Jun. 1996.